Informe Técnico: Plan de Trabajo para la Construcción del Software

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Este informe especifica las herramientas y tecnologías de versionamiento que se emplean en la construcción de **MONK GRAIN**, una plataforma de compra y venta de café pergamino seco en Colombia que conecta a caficultores, fincas y cooperativas con compradores por volumen —trilladoras, exportadoras y tostadoras—.

La selección no es un ejercicio teórico: las herramientas que aquí se describen son las que el proyecto usa hoy, sobre un repositorio con 13 versiones registradas y una canalización de integración continua de tres trabajos. Donde una decisión se tomó por una razón concreta del proyecto, esa razón queda escrita, porque un plan de trabajo que solo enumera herramientas no explica por qué esas y no otras.

2. Objetivo

Especificar las herramientas y tecnologías de versionamiento que se utilizarán en la construcción del software, de acuerdo con las características del proyecto MONK GRAIN: su tamaño, la cantidad de lenguajes que coexisten, la necesidad de reproducir el entorno en cualquier equipo y la exigencia de poder volver atrás sin perder trabajo.

3. Características del software que condicionan la elección

Las herramientas se eligen por lo que el software es, no al revés. Estas son las características que pesaron en la decisión:

Característica	Medida	Qué exige del versionamiento
Dos lenguajes en un mismo repositorio	99 archivos Python · 83 TypeScript	Una herramienta agnóstica del lenguaje, que no imponga un ecosistema
Base de datos con esquema versionado	54 tablas · 7 migraciones	Las migraciones viajan con el código: el esquema es parte de la versión
Suite de pruebas amplia	453 pruebas de backend · 41 de frontend	Verificación automática en cada cambio, no a mano
Entorno reproducible	6 servicios en contenedores	La definición del entorno se versiona junto al código
Trabajo por fases largas	13 versiones registradas	Ramas para aislar el trabajo en curso de lo que ya funciona

4. Herramienta de control de versiones: Git

Se selecciona **Git** como sistema de control de versiones. Las razones, en orden de peso para este proyecto:

4.1 Es distribuido

Cada copia del repositorio contiene el historial completo. En un proyecto formativo que se trabaja desde un equipo personal y sin conexión permanente, poder registrar versiones, consultar el historial y crear ramas sin servidor es determinante. Un sistema centralizado como Subversion exigiría estar conectado para cada operación.

4.2 Es agnóstico del lenguaje

MONK GRAIN combina Python, TypeScript, SQL, HTML y archivos de configuración en un mismo árbol. Git versiona archivos, no proyectos de un lenguaje concreto, de modo que una sola herramienta cubre todo el repositorio.

4.3 Las ramas son baratas

Crear una rama en Git es escribir un archivo con un identificador, no copiar el proyecto. Eso permite aislar un trabajo largo sin bloquear lo que ya funciona. En este proyecto se usa ahora mismo: la rama `main` conserva la última versión estable y `correcciones-auditoria` contiene las correcciones en revisión.

```
$ git branch
  correcciones-auditoria
* main

$ git log --oneline -3
e02eed6 El porcentaje de la comisión, con su espacio
360ea1d Auditoría del estado y corrección de sus hallazgos
00db999 Rastros del producto anterior en la interfaz
```

4.4 Permite volver atrás con precisión

Cada versión registra qué cambió, quién lo hizo y cuándo. Ante un error introducido, `git revert` deshace un cambio concreto sin borrar el historial, y `git bisect` localiza en qué versión apareció un defecto buscando por bisección en lugar de revisando una por una.

5. Plataforma de alojamiento remoto: GitHub

Git resuelve el versionamiento local. El respaldo fuera del equipo y la entrega al instructor necesitan además un alojamiento remoto. Se selecciona **GitHub**:

Criterio	Razón de la elección
Respaldo fuera del equipo	El historial deja de depender de un solo disco. Sin remoto, un fallo del equipo borra el proyecto completo y su historia
Entrega de las evidencias	Las guías GA7 y GA8 exigen el enlace del repositorio en cada evidencia; una URL pública lo resuelve
Integración continua incluida	GitHub Actions ejecuta las pruebas en cada envío sin contratar otro servicio
Revisión por diferencias	Las pull requests muestran el cambio línea a línea antes de integrarlo

6. Herramientas complementarias del flujo de versionamiento

6.1 Integración continua: GitHub Actions

El archivo `.github/workflows/ci.yml` define tres trabajos que se ejecutan en cada envío a `main` o `develop` y en cada pull request:

Trabajo	Qué verifica
Backend	Linting y formato (`ruff`), verificación de tipos (`mypy`), aplicación de migraciones, comprobación de que no falten migraciones (`alembic check`) y pruebas con cobertura
Frontend	Verificación de tipos (`tsc`), linting (`eslint`), formato, pruebas y compilación de producción
Docker	Que las imágenes de producción se construyan

La verificación de que no falten migraciones merece mención aparte: detecta el olvido más común del proyecto —cambiar un modelo sin generar la migración correspondiente—, que de otro modo solo aparecería al desplegar.

6.2 Reproducibilidad del entorno: Docker y archivos de versiones fijas

Versionar el código no basta si cada equipo instala versiones distintas de las librerías. El proyecto versiona también la definición del entorno:

- `docker-compose.yml` y los `Dockerfile`, que describen los seis servicios.
- `backend/requirements.lock`, con las 78 versiones exactas de Python, incluidas las transitivas.
- `web/package-lock.json`, con el equivalente para Node.

> **Por qué los archivos de versiones fijas.** Las dependencias estaban declaradas solo con `>=`, y una reconstrucción de la imagen trajo versiones mayores nuevas sin avisar: SQLAlchemy 2.1 cambió la firma de `select()` y rompió la verificación de tipos de nueve archivos a la vez. Fijar las versiones hace que el entorno se reproduzca igual hoy y dentro de seis meses.

6.3 Convención de mensajes de versión

Un historial solo sirve si se puede leer. Los mensajes de este proyecto siguen una convención: una primera línea corta que dice qué cambió en lenguaje llano, y un cuerpo que explica por qué. El detalle está en la evidencia AA1-EV02.

7. Flujo de trabajo adoptado

1. **Rama por trabajo.** Cada fase o corrección se desarrolla en su propia rama, nunca directamente sobre `main`.
2. **Versiones pequeñas y con sentido.** Un cambio por versión, con su mensaje explicando el motivo.

3. **Verificación antes de integrar.** La canalización debe pasar en verde: linting, tipos, migraciones y pruebas.
4. **Integración a `main`.** Solo cuando la rama está verificada.
5. **Envío al remoto.** Tras cada integración, para que el respaldo no se retrase.

8. Resumen de la selección

Necesidad	Herramienta	Versión / referencia
Control de versiones local	Git	2.x
Alojamiento remoto	GitHub	github.com
Integración continua	GitHub Actions	`.github/workflows/ci.yml`
Reproducibilidad del entorno	Docker y Docker Compose	`docker-compose.yml`
Versiones fijas de Python	Archivo de restricciones de pip	`backend/requirements.lock`
Versiones fijas de Node	npm	`web/package-lock.json`
Calidad verificada en cada envío	ruff · mypy · eslint · tsc · pytest · vitest	Configurados en la canalización

9. Conclusión

Las herramientas seleccionadas responden a las características concretas de MONK GRAIN: Git por ser distribuido y agnóstico del lenguaje, en un repositorio donde conviven Python y TypeScript; GitHub por el respaldo fuera del equipo y la entrega de las evidencias; GitHub Actions por verificar cada cambio sin intervención manual; y Docker junto a los archivos de versiones fijas porque versionar el código sin versionar el entorno deja el proyecto a merced de lo que cada equipo tenga instalado.

El conjunto ya está en funcionamiento sobre el repositorio del proyecto, y es el que sostiene las evidencias siguientes de esta guía.